

Enterprise Multi-Agent Knowledge Assistant

A self-hosted question-answering platform over an organisation's documents. Answers are produced by an eight-agent pipeline, carry citations and a confidence score, and respect department-level access control enforced in code.

BACKEND

Python 3.12 · FastAPI · LangGraph

FRONTEND

Next.js 16 · React 19 · TypeScript

DATA

PostgreSQL 16 · Qdrant (SQLite locally)

CLIENTS

Web · Windows · macOS · Android

VERIFIED

108 automated backend tests passing

SCOPE

36 API operations · 9 tables · 8 agents

Gopi Patel

Portfolio project · AI-assisted development · September 2026

Figures in this report were measured from the codebase and test runs at the time of writing.

Contents

1. Project Overview	3
2. Project Objectives	3
3. Problem & Requirements	4
4. Solution Overview	4
5. Key Features & Capabilities	5
6. System Architecture	6
7. Technology Stack	7
8. Core Workflows	8
9. Important Engineering Decisions	9
10. AI-Assisted Development	10
11. Implementation Details	11
12. Challenges & Solutions	12
13. Testing & Verification	13
14. Security, Reliability & Data Integrity	14
15. Deployment & Usage	15
16. My Role & Contribution	15
17. Results & Verified Metrics	16
18. Limitations	16
19. Future Improvements	17
20. Conclusion	17
Appendix A – Interface Walkthrough	18
Appendix B – API Reference	22
Appendix C – Master Prompt	23

1. Project Overview

The Enterprise Multi-Agent Knowledge Assistant lets an organisation ask questions of its own documents. Administrators upload policies, handbooks, spreadsheets and slide decks; employees ask questions in plain language and receive an answer that names the document, department, version and page it came from, together with a confidence score.

Each organisation runs its own backend, so documents stay on its own infrastructure. The same user interface is delivered as a web application, a desktop application for Windows and macOS, and an Android application. The packaged apps contain only the interface and connect to the organisation's server over HTTP.

Audience	What they do
Employees	Ask questions, inspect citations and confidence, watch the agent pipeline work in real time, keep and rename conversation history, rate answers, and manage their own sign-in email and password.
Administrators	Upload, version, re-index, export and delete documents; invite or create users and set their role and department; review usage analytics, token consumption, the audit trail and every conversation (read-only). Administrators do not use the chat themselves.

Current status

Area	State
Backend, agent pipeline, ingestion	Complete and covered by 108 passing tests.
Web interface (both portals)	Complete; type-checks, lints and builds cleanly.
Windows desktop installer	Built on Windows and used there; includes the credential-management screens and the built-in server address.
macOS installer (Apple Silicon)	Built earlier; launches after removing the download quarantine flag; predates the latest interface changes.
Android APK	Built earlier; must be rebuilt to include the cleartext-HTTP fix and the latest interface (see Limitations).
iOS	Configured in CI only; never built or tested.

2. Project Objectives

Findability. Make knowledge scattered across PDF, Word, PowerPoint, Excel, text and Markdown files retrievable by meaning, not file name.

Grounded answers. Every answer is traceable to specific document chunks; the system says so plainly when the evidence does not answer the question.

Hallucination control. Verify evidence before answering and review the answer afterwards, with bounded retries rather than unbounded loops.

Access control that cannot be talked around. Department permissions are applied in code before retrieval, independent of anything the model outputs.

Accountability. Record logins, document changes, user changes, invitations and every query in an append-only audit log, with token usage and latency analytics.

Vendor flexibility. Keep the language-model and embedding providers behind interfaces so a provider change is one new class.

Offline development. Run and test the full pipeline with no API key, database server or network access.

One codebase, several clients. Ship web, desktop and mobile clients that can be pointed at any organisation's server.

3. Problem & Requirements

Organisational knowledge lives in many files that are hard to search, and a language model connected to those files naively will answer confidently from the wrong source, or from none. The system therefore had to retrieve the right material, prove where each answer came from, and never expose a document to someone outside the department that owns it.

Functional requirements

Requirement	How it is met
Upload six document formats	Loaders for PDF, DOCX, TXT, MD, PPTX and XLSX that keep page, slide or sheet numbers.
Answers with sources	Generator cites numbered evidence; citations are assembled in code from the evidence actually cited.
Confidence per answer	Verifier and reflection agents produce 0–1 scores, shown as a colour-coded badge.
Department isolation	Policy agent computes permitted departments; Qdrant search is filtered by them.
Document lifecycle	Upload, replace with version bump, re-index, export in five formats, delete (row, file and vectors).
Account management without sign-up	Single-use invitation links or direct creation; admin and self-service email/password changes.
Separate portals	Employee and Administrator portals with guards in both directions.
Oversight	Chat logs, analytics (queries, tokens, latency, confidence, feedback) and a filterable audit log.
Live progress	Server-Sent Events report each agent as it finishes.

Non-functional requirements

- Typed end to end: Pydantic schemas on the API, strict TypeScript mirrors in the client.
- Async I/O throughout the backend.
- Tests runnable with no external services, using SQLite, in-memory Qdrant and fake providers.
- Deployable with Docker Compose; refuses to start in production with the default JWT secret.
- Runnable on a Windows workstation by double-clicking one file, and reachable from other devices on the same network.

4. Solution Overview

A FastAPI backend holds all business logic in a service layer over repositories. Questions are answered by a LangGraph state machine of eight agents. Three of them are deterministic code (access policy, retrieval and evidence selection); five call the language model with schema-validated outputs. Uploaded documents are processed in the background into Qdrant, with PostgreSQL as the system of record. A Next.js interface serves two portals and is also packaged into Electron and Capacitor shells.

Component	Responsibility
api/	HTTP routing, request validation, dependency injection, auth guards.
services/	Business rules: auth, users, invitations, documents, export, chat, storage, vectors.

Component	Responsibility
repositories/	Data access; never commit, so each request is one transaction.
graph/	The eight agents, shared state, provider protocols; no database access.
ingestion/	Loaders, chunker, embedding providers, background pipeline.
frontend/	Next.js 16 app, typed API client, auth context, runtime server configuration.
desktop/	Electron shell serving the static UI from a local HTTP server.
frontend/android/	Capacitor shell for Android.
setup / start scripts	One-step Windows setup and a double-click start in network mode.

5. Key Features & Capabilities

Capability	Detail
Cited answers	Document name, department, version, page and a snippet for every citation.
Confidence scoring	Badge: green at 70% and above, amber from 40%, red below.
Live agent pipeline	Side panel shows each of the eight agents as pending, running or done, with its result and a retry counter.
Department RBAC	Own department + general + granted departments; administrators see all.
Document versioning	Replacing a file bumps the version; old vectors are removed first; evidence selection keeps only the newest version.
Export	Original file, or PDF, DOCX, TXT and Markdown generated from the extracted text.
Invitations	Link shown once, stored only as a SHA-256 hash, single use, expiring, revocable; role and department fixed by the invite.
Credential management	Admins change any user's email or password; users change their own after re-entering the current password.
Conversation history	Threaded conversations, rename, save, delete, per-message feedback.
Oversight	Org-wide read-only chat logs, usage analytics and audit trail.
Configurable server	Packaged apps can be pointed at any server from the sign-in screen, or ship with an address built in.
Multi-device access	One double-click starts the servers in network mode; phones and other computers use the apps or a browser.
Provider independence	OpenAI and deterministic fake providers behind protocols.

6. System Architecture

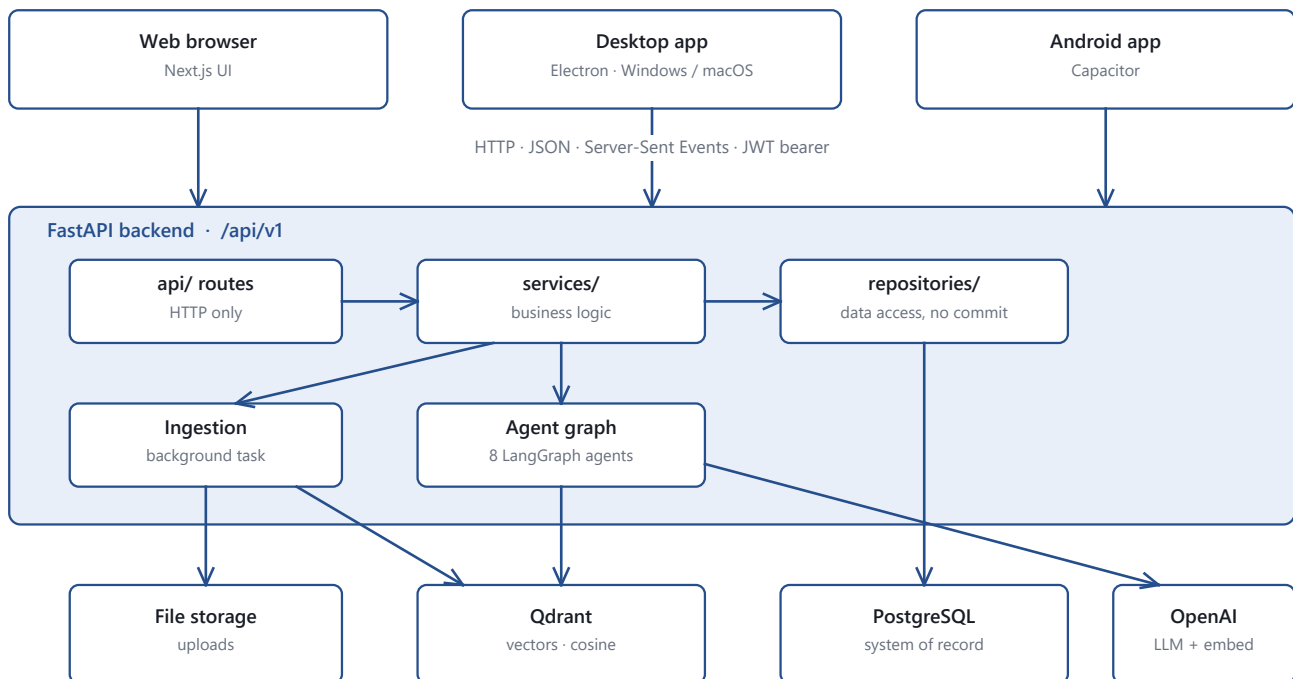


Figure 1 – System architecture. Locally, PostgreSQL is replaced by SQLite and the Qdrant server by its embedded engine; the fake LLM and embedding providers replace OpenAI. Only configuration changes between the two.

Layering rules

- Routes contain no business logic; they validate input, resolve dependencies and call a service.
- Services own transactions' content; repositories only query and add, so a failure anywhere rolls the whole request back.
- The agent graph receives its dependencies (LLM, embedder, vector store, settings) through a GraphDeps object and never opens a database session.
- The application is built by `create_app(settings)`; tests construct isolated instances with their own settings.
- Errors are raised as typed `AppError` subclasses and rendered as one JSON envelope: `{"error": {"code", "message"}}`.

Data model

Table	Purpose	Notable fields
users	Accounts	email (unique, lower-cased), role, department, is_active
documents	Uploaded files	version, status, chunk_count, error_message
conversations	Chat threads	user_id, title, is_saved
chat_messages	Messages	citations (JSON), confidence, token_usage (JSON)
feedback	Ratings	one per user and message (upsert)
permissions	Extra department grants	unique (user_id, department)
audit_logs	Append-only trail	action, resource, details (JSON), ip_address
analytics_events	Usage and cost	prompt/completion tokens, latency_ms
invitations	Pending accounts	token_hash, expires_at, accepted_at

Qdrant stores one point per chunk in a cosine-distance collection with keyword indexes on department and document_id. The payload carries document id and name, department, version, chunk index, page, source file name, upload time and text, which is everything a citation needs without a database round trip.

7. Technology Stack

Versions below are those installed and verified in the working environment.

Layer	Technology	Why
Runtime	Python 3.12.10 · Node.js 24	Current language features; async support.
API	FastAPI 0.141 · Uvicorn 0.52 · Pydantic 2.13	Typed async API with generated OpenAPI schema.
Agents	LangGraph 1.2	Explicit state machine with conditional edges and streaming.
Persistence	SQLAlchemy 2.0 (async) · Alembic 1.19 · asyncpg /aiosqlite	One model layer for PostgreSQL and SQLite; versioned migrations.
Vectors	qdrant-client 1.19	Same client for server, embedded-on-disk and in-memory modes.
Models	openai 3.7 (gpt-4o-mini, text-embedding-3-small by default)	Structured outputs; swappable.
Security	PyJWT · bcrypt 5	Signed access/refresh tokens; salted password hashes.
Ingestion	pypdf · python-docx · python-pptx · openpyxl · langchain-text-splitters	Format loaders with provenance; recursive chunking.
Export	fpdf2 · python-docx	PDF and Word generation from extracted text.
Logging	structlog	Readable in development, JSON elsewhere.
Frontend	Next.js 16.2 · React 19.2 · TypeScript 5 · Tailwind 4	App Router; static export for packaged apps.
UI kit	shadcn/ui on Base UI · next-themes · lucide-react · sonner	Accessible components, dark mode, toasts.
Clients	Electron 33 + electron-builder · Capacitor 8	Desktop installers and Android APK from one UI.
Delivery	Docker Compose · GitHub Actions	Containerised stack; per-OS release builds.
Testing	pytest 9 · pytest-asyncio · httpx	Full API tests without external services.

8. Core Workflows

Answering a question

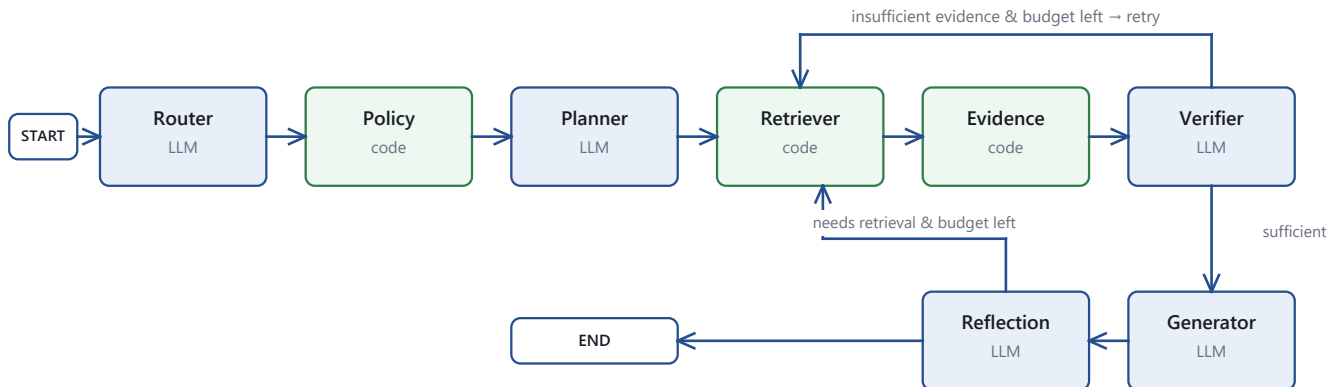


Figure 2 – Agent graph. Green nodes are deterministic code; blue nodes call the language model. Both retry loops draw from one shared iteration budget (`MAX_AGENT_ITERATIONS`, default 2), so the graph always terminates.

Agent	Kind	Responsibility
Router	LLM	Pick one or two departments likely to hold the answer; "general" when unclear.
Policy	Code	Compute the departments the user may read and clamp the router's choice to them.
Planner	LLM	Write a short plan and one to three search queries.
Retriever	Code	Embed the queries and run department-filtered vector search (top 8).
Evidence	Code	Keep the newest document version, drop weak hits, dedupe, keep top 6.
Verifier	LLM	Judge whether evidence is sufficient and consistent; propose a better query if not.
Generator	LLM	Answer only from numbered evidence with [n] markers.
Reflection	LLM	Check completeness and citations; set the final confidence; may request one more round.

With `/chat/stream` the browser receives one Server-Sent Event per completed agent (for example the departments chosen, the number of hits and the retrieval round), then the final message with citations and confidence. The same service method backs `/ask` and `/chat`, so audit and analytics records are written in one place.

Publishing a document

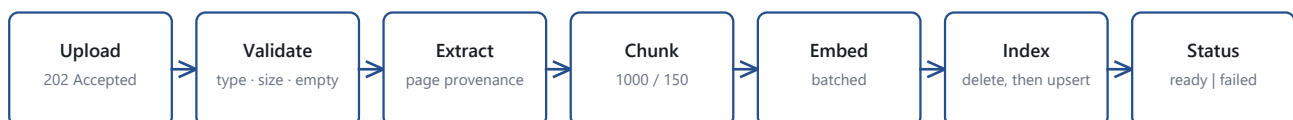


Figure 3 – Ingestion pipeline. Steps after `Validate` run as a background task with its own database session; the document row records progress.

- The upload request validates and stores the file, creates the document row as pending, schedules ingestion and returns 202 immediately.
- The background task marks the row processing, extracts sections with page numbers, splits them into 1,000-character chunks with 150 overlap, embeds them, deletes any existing vectors for that document and upserts the new ones.
- Success sets status ready with the chunk count; any exception sets failed and stores the message, which the Documents page shows. The page polls while any row is pending or processing.

Adding a user by invitation

- An administrator chooses email, role, department and validity (1–90 days, default 7).
- The server generates a random token, stores only its SHA-256 hash, and returns the link once; re-inviting the same address replaces the earlier link.
- The invitee opens the link, the server confirms it is unused and unexpired, the invitee sets a password, and the account is created with the role and department from the invitation. They are signed in immediately; the invitation is marked accepted.

9. Important Engineering Decisions

Each decision is stated as Problem → Decision → Implementation → Result.

Access control outside the model

Problem A prompt instruction cannot be relied on to keep finance documents from an HR employee.

Decision Make the Policy agent plain code that runs before retrieval.

Implementation `compute_allowed_departments()` returns own + general + granted (all for admins); the router's choice is intersected with it and the result filters the Qdrant query.

Result Unauthorised chunks are never fetched, so they cannot reach the prompt or a citation. Tested: an HR employee receives no finance content; an administrator reaches all.

Treat "general" as a catch-all and widen on retry

Problem A vaguely worded question was routed to "general"; narrowing to general-tagged documents skipped the user's own department, and the retry repeated the same search.

Decision Widen to every permitted department when "general" is chosen, and on any retry round.

Implementation Policy replaces the permitted set with the allowed set in both cases; the retriever uses `allowed_departments` whenever `iteration > 0`.

Result Vague questions still find the user's documents; the permission ceiling is unchanged. Covered by two graph tests.

One budget for both loops

Problem Two independent retry loops could keep calling the model.

Decision Share one iteration counter between the verifier and reflection loops.

Implementation The retriever increments iteration; both conditional edges compare it with `MAX_AGENT_ITERATIONS` (default 2).

Result The graph always terminates; with an empty knowledge base it loops once and degrades to a plain "not found" answer (tested).

Deterministic evidence handling and citations

Problem Ranking and citation formatting done by a model add cost and can invent sources.

Decision Select evidence in code; let the model return only the indexes it cited.

Implementation `select_evidence()` keeps the newest version, applies a score floor, dedupes and sorts; the generator builds Citation objects from the returned indexes, ignoring out-of-range ones.

Result Citations always correspond to real retrieved chunks.

Providers behind protocols, with fakes

Problem Tests and local development should not need an API key, and the vendor may change.

Decision Define LLMClient and EmbeddingProvider protocols with OpenAI and fake implementations.

Implementation `LLM_PROVIDER` / `EMBEDDING_PROVIDER` select the implementation; the fake LLM reads marker lines the prompts already include.

Result All 108 tests run offline. Adding another vendor is one class.

Background ingestion with an honest status

Problem Embedding a large file inside the upload request would time out and hide failures.

Decision Return 202 and ingest in a background task that owns its session.

Implementation The document row moves pending → processing → ready | failed and stores the error.

Result Uploads are responsive and failures are visible to administrators.

Invitations instead of self-registration

Problem Department assignment is what the policy enforces, so users must not choose it.

Decision Administrators issue single-use links whose role and department are fixed.

Implementation Only a SHA-256 of the token is stored; expiry, single use and revocation are checked on preview and accept.

Result A leaked database does not leak usable links; invitees cannot raise their own access.

Server address decided at runtime

Problem One packaged app must work against any organisation's server.

Decision Resolve the API base at runtime: saved choice, else the build-time default.

Implementation The sign-in screen offers Change / Test & save (probing /health); a build can bake in an address; a loopback default is replaced with the page's host when the web UI is opened from another device.

Result The same installer serves different deployments; phones can use the web UI directly.

Client-side timestamps for append-only tables

Problem Audit and analytics inserts failed on SQLite with "unknown function: now()".

Decision Supply timestamps from Python and write migration defaults dialect-neutrally.

Implementation default=datetime.utcnow on both models; migrations use sa.func.now() instead of sa.text('now()').

Result Existing databases work without rebuilding; new databases get CURRENT_TIMESTAMP on SQLite and now() on PostgreSQL.

10. AI-Assisted Development

The codebase was produced with AI coding assistants working under my direction. The assistants wrote most of the code; I defined what to build, decided between options, tested the results on real devices and reported what did not work. The working record for the later stages of the project (porting, packaging, defect fixes and new features) is the development session in which this report was prepared.

Stage	How AI was used	My part
Specification	A detailed build specification (docs/MASTER_PROMPT.md) guided a milestone-by-milestone build.	Product scope, portals, access rules and quality bar, as captured in the specification.
Implementation	The assistant wrote backend, agent, frontend, packaging and script code, with tests for each milestone.	Reviewed milestones and approved each before the next.
Porting to Windows	Diagnosed macOS-only artefacts (virtualenv, native Node binaries, POSIX build script) and replaced them.	Required the project to run on my Windows machine.
Packaging	Built the Windows installer, worked around a symlink-privilege failure, baked in the server address.	Chose one server address for all apps and the platforms to keep.
Debugging	Traced reported failures to root causes: SQLite now() defaults, Android cleartext, macOS quarantine, loopback binding, CORS.	Found them by using the apps on Windows and macOS and sent screenshots.

Stage	How AI was used	My part
Features	Added credential management and multi-device start-up, with tests.	Asked for the features and the no-prompt, any-device start behaviour.
Verification	Ran the test suite, type-checks, lint, builds, live HTTP checks and packaged-app launches after each change.	Confirmed behaviour on real devices.

How verification was handled

Changes were accepted only after an observable check: the backend suite (108 passing), TypeScript and ESLint on changed frontend files, production builds, requests against the running server (for example upload → ready with 100 chunks, /ask returning 3 citations), and launches of the packaged app. When a check failed, the failure was reported and fixed rather than skipped.

11. Implementation Details

Access policy (graph/nodes/policy.py)

```
def compute_allowed_departments(*, role, department, granted):
    if role == UserRole.ADMIN.value:
        return [d.value for d in Department]
    return sorted({department, Department.GENERAL.value, *granted})

permitted = [d for d in routed if d in allowed]
if not permitted:
    permitted = allowed          # router chose only forbidden departments
elif Department.GENERAL.value in permitted:
    permitted = allowed          # "general" is a catch-all, not a filter
```

Retry widening (graph/nodes/retriever.py)

```
is_retry = state.get("iteration", 0) > 0
scope = (state.get("allowed_departments", []) if is_retry
        else state.get("routed_departments", []))
...
return {"retrieved": hits, "iteration": state.get("iteration", 0) + 1}
```

Evidence selection (graph/nodes/evidence.py)

```
newest = {}                                # newest version per document
for hit in hits:
    newest[hit.document_id] = max(newest.get(hit.document_id, 0), hit.version)
best = {}
for hit in hits:
    if hit.version < newest[hit.document_id] or hit.score < min_score:
        continue
    key = (hit.document_id, hit.chunk_index)
    if key not in best or hit.score > best[key].score:
        best[key] = hit
return sorted(best.values(), key=lambda c: c.score, reverse=True)[:top_k]
```

Server address resolution (frontend/src/lib/config.ts)

```
export function getApiBase(): string {
  if (typeof window === "undefined") return BUILD_TIME_BASE;
  return localStorage.getItem(STORAGE_KEY) || reachableFromHere(BUILD_TIME_BASE);
}
// A loopback default means "this machine". Opened from a phone at
// http://192.168.x.x:3000, point at the host the page came from instead.
```

Packaged clients

Client	Implementation
Desktop (Electron)	main.js serves the static export from an HTTP server on 127.0.0.1 (random port, path-traversal guard, SPA fallback) instead of file://, so routing and storage behave as in a browser. External links open in the system browser.
Build sync	sync.mjs builds the frontend with APP_SHELL=1 and copies the export into the shell. It is Node rather than shell so it runs on Windows, and it refuses to run while a dev server is using the shared .next directory.
Android (Capacitor)	The same export runs in a WebView. The manifest allows cleartext HTTP because organisation servers are often plain HTTP on a private network.
Windows tooling	setup.cmd creates the environment idempotently; start.cmd starts both servers in network mode, checks the PC's address against the one built into the apps, opens the firewall once with administrator approval, and stops whole process trees on exit.

12. Challenges & Solutions

Symptom	Cause	Solution
Every document upload failed on the local database	Migrations wrote the literal default now(), which SQLite lacks; audit and analytics rows relied on it.	Python-side timestamps on those models; migrations switched to sa.func.now().
Android app could not reach an http:// server	allowMixedContent was set, but Android's cleartext policy (target SDK 36) was never relaxed.	usesCleartextTraffic="true" in the manifest. Rebuild pending.
macOS: "Knowledge Assistant is damaged"	Unsigned app plus the browser's quarantine attribute.	Copy out of the disk image and remove the attribute; documented.
Mac and phones could not connect	Backend bound to 127.0.0.1; firewall closed; web UI pointed at localhost; CORS and Next.js dev checks allowed only localhost.	Network mode binds 0.0.0.0; one-time firewall rule; host-relative API default; private-network CORS; allowedDevOrigins.
Windows installer build failed	electron-builder's signing-tool archive contains macOS symlinks; creating symlinks needs administrator rights or Developer Mode.	Used the otherwise complete extraction under the expected cache name; three fixes documented.
Project would not run after moving from macOS	POSIX virtualenv, darwin-only Node binaries, POSIX build script.	Rebuilt environments; replaced the build script with Node; added setup/start scripts.
Second backend crashed at start-up	The embedded Qdrant store allows one process.	start script detects a running backend and explains how to stop it.
Ports stayed busy after stopping	uvicorn --reload and next dev hand the socket to a grandchild process.	Kill process trees, then sweep processes whose command line contains the project path.

Symptom	Cause	Solution
Backend took over 30 s to start	The reloader watched the whole backend folder, including the virtualenv.	--reload-dir app; start-up fell to a few seconds.
Vague questions found nothing	"general" routing narrowed the search; retries repeated it.	Widening rules in policy and retriever (Section 9).

Why tests missed the timestamp defect

The test suite builds its schema from the models with `create_all()`, where `func.now()` is rendered correctly for SQLite. Only a database built by the migrations – exactly what the setup script creates – carried the invalid default. The fix was verified against a copy of the real database, which still contains the old default, and against a fresh migration run (0 invalid defaults, 9 tables using `CURRENT_TIMESTAMP`).

13. Testing & Verification

The backend suite runs against a file-based SQLite database created per test, an in-memory Qdrant instance and the fake language-model and embedding providers. No database server, vector server, API key or network access is needed.

Suite	Tests	What it checks
test_health	2	Start-up; unknown routes return 404.
test_db	4	Models, relationships, table registration, readiness probe.
test_auth	8	Token pair, wrong password and unknown email, /me, refresh, token-type confusion, malformed tokens.
test_users	6	Admin-only access, create and sign in, duplicate email, deactivation, password length.
test_user_credentials	12	Admin email/password changes, duplicate emails, cross-user attempts, self-service changes, wrong current password (422, not 401), audit without secrets.
test_invitations	11	Permissions, accept and sign in, role/department from invite, single use, unknown and expired tokens, existing users, re-invite, revoke rules.
test_documents	13	Upload→ready, permissions, format and empty-file rejection, filters, versioning, re-index, all five downloads, delete.
test_graph	8	All eight agents; cited answers; vague-query and retry widening; RBAC; admin reach; empty knowledge base.
test_ask	5	Auth, cited answers, HR/finance isolation, admin reach, validation.
test_chat	11	Persistence, listing, ownership, rename/save/delete, feedback, SSE events, continued threads, admin chat logs, analytics, audit logs.
test_enum_migrations	28	Every persisted enum value appears in a migration (parametrised; guards a PostgreSQL-only failure).

Result

108 passed (80 test functions plus 28 parametrised enum cases), measured on the final code; runs took 133–171 seconds on the development machine.

Other verification performed

Check	Result
Frontend	tsc --noEmit and ESLint clean on changed files; production build of 12 routes succeeded.
Live ingestion	A 1.9 MB PDF uploaded through the API reached READY with 100 chunks.
Live question	/ask returned 200 with 3 citations and 0.8 confidence (fake providers).
Network access	API and web UI reachable on the LAN address; CORS pre-flight from a LAN origin allowed; administrator sign-in over the LAN returned 200.
CORS rule	13 origin cases: private and loopback allowed; public and look-alike hosts rejected.
Windows start-up	start script brought both servers up with no input in about 14 s.
Windows app	The packaged app (win-unpacked) launched and rendered the sign-in screen; the baked address was present in the bundle and localhost absent. The installer was also installed and used on Windows.

Not covered: there are no automated frontend component or end-to-end browser tests, and answer quality with the OpenAI providers has not been measured.

14. Security, Reliability & Data Integrity

Concern	Measure
Passwords	bcrypt hashes; minimum length 8; the same message for unknown email and wrong password, with timing parity via a dummy hash.
Tokens	HS256 JWTs with type and unique id claims; access 30 min, refresh 7 days; each type rejected where the other is expected.
Session validity	Inactive users are rejected on every request, not only at sign-in.
Authorisation	Admin-only routes; owners only for conversations and feedback (others get 404, so existence is not revealed); department policy before retrieval.
Credential changes	Self-service changes require the current password; audit entries record which fields changed and by whom, never the values.
Invitations	Only token hashes stored; single use; expiry; revocation; no self-chosen role.
Uploads	Extension, size and emptiness checks; UUID file names; client path components stripped; storage guards against path escape.
Transactions	One transaction per request; repositories never commit; ingestion uses its own session and records failures.
Index consistency	Vectors deleted before re-upsert; evidence restricted to the newest version.
Schema safety	A static test ensures new enum values have PostgreSQL migrations.
Production start-up	Refuses the default JWT secret; API docs disabled; compose fails if required secrets are unset; containers run as non-root.
Network exposure	CORS allows loopback and private networks only; the firewall rule applies to private and domain networks, not public ones.

15. Deployment & Usage

On a Windows workstation

```
setup.cmd      one-time: Python environment, dependencies, migrations, frontend packages
start.cmd      every time: both servers in network mode, browser opens

Other devices on the same network:
  browser  ->  http://<this-pc-ip>:3000
  apps     ->  server http://<this-pc-ip>:8000
```

The local configuration uses SQLite, embedded Qdrant and the fake providers, so it runs without external services; answers are placeholders until OpenAI providers and a key are configured and documents are re-indexed. On the first network-mode start, Windows asks once for permission to open ports 8000 and 3000.

With Docker Compose

Service	Configuration
postgres	postgres:16-alpine; pg_isready health check gates the backend; named volume.
qdrant	Qdrant server v1.13.4; named volume. Switching from embedded mode is a QDRANT_URL change.
backend	Multi-stage image, non-root, curl health check, two Uvicorn workers. The entrypoint waits for the database, runs migrations and can seed the first administrator.
frontend	Three-stage Node 22 image producing the standalone bundle; non-root. The API URL is a build argument because it is compiled into the client.

Client builds

Target	Command / pipeline	Notes
Windows	desktop: npm run build:win	NSIS installer, chooses install folder, desktop shortcut.
macOS	desktop: npm run build:mac	Apple Silicon and Intel disk images; macOS host required.
Linux	desktop: npm run build:linux	Applimage.
Android	Capacitor sync + Gradle assemble	APK.
All	GitHub Actions release workflow	Builds each platform on its own OS and publishes a release.

Setting NEXT_PUBLIC_API_BASE at build time bakes a server address into the installers; users can still change it on the sign-in screen.

16. My Role & Contribution

I owned the product and its direction; implementation was carried out with AI coding assistants, as described in Section 10. The evidence for my contribution is the specification, the sequence of requirements and defect reports in the development record, and testing on my own devices. The main repository has no commit history yet, so authorship of individual lines is not recorded in version control.

Product definition. Chose the problem, the two-portal model, department-based access and the requirement for cited, confidence-scored answers.

Delivery decisions. Required the system to run on Windows, chose one server address for all apps, one macOS build (Apple Silicon), one start file with no prompts, and access from any device on my network.

Feature requests. Email and password management for administrators and users.

Testing on real devices. Installed and used the Windows and macOS apps; reported the macOS quarantine error, connection failures between devices, and the document-upload failure with screenshots, which led to the fixes in Section 12.

Acceptance. Reviewed results and decided what to keep, rebuild or remove (for example the release set and the redundant start script).

Build environment. Provided the Windows machine used for builds, with JDK 17, the Android SDK and Node.js already installed.

17. Results & Verified Metrics

Metric	Value	Source
Backend tests	108 passing	pytest run on the final code
Test functions	81 in 11 suites (one parametrised into 28 cases)	tests/ directory
API	27 paths, 36 operations	Generated OpenAPI schema
Database tables	9	SQLAlchemy metadata
Migrations	3	alembic/versions
Agents	8 (3 deterministic, 5 model-backed)	graph/builder.py
Backend source	87 files, 5,527 lines	app, scripts, alembic
Backend tests source	13 files, 1,669 lines	tests
Frontend source	36 files, 5,377 lines; 11 pages; 16 UI primitives	frontend/src
Desktop shell	209 lines	main.js, sync.mjs
Windows scripts	445 lines	setup/start .ps1 and .cmd
Input / export formats	6 / 5	loaders, export service
Release artefacts	Windows .exe (current), macOS .dmg (arm64), Android .apk	release/

No answer-accuracy or latency benchmarks with real models were run, so none are reported.

18. Limitations

- The released Android APK predates the cleartext fix and the latest interface, so it cannot reach an http:// server until it is rebuilt; a rebuild on Windows was started but not completed.
- The macOS disk image is Apple Silicon only and predates the credential-management screens.
- Installers are unsigned: Windows SmartScreen and macOS Gatekeeper warn on first launch.
- iOS has never been built or tested.
- With the fake providers, answers are placeholder text; real quality needs an OpenAI key and a re-index.
- The embedded Qdrant engine allows one backend process at a time.
- Converted exports keep text only; PDF export uses Latin-1 core fonts, so other scripts degrade.
- Refresh tokens are kept in browser storage rather than httpOnly cookies.
- Network mode exposes a development backend (debug on, API docs, seeded accounts) to the local network.
- The server address built into the apps is a DHCP-assigned address unless reserved on the router.
- Existing local databases still contain the old now() default in their schema (unused by the application).
- There are no automated frontend tests, and the main repository has no commits yet.

19. Future Improvements

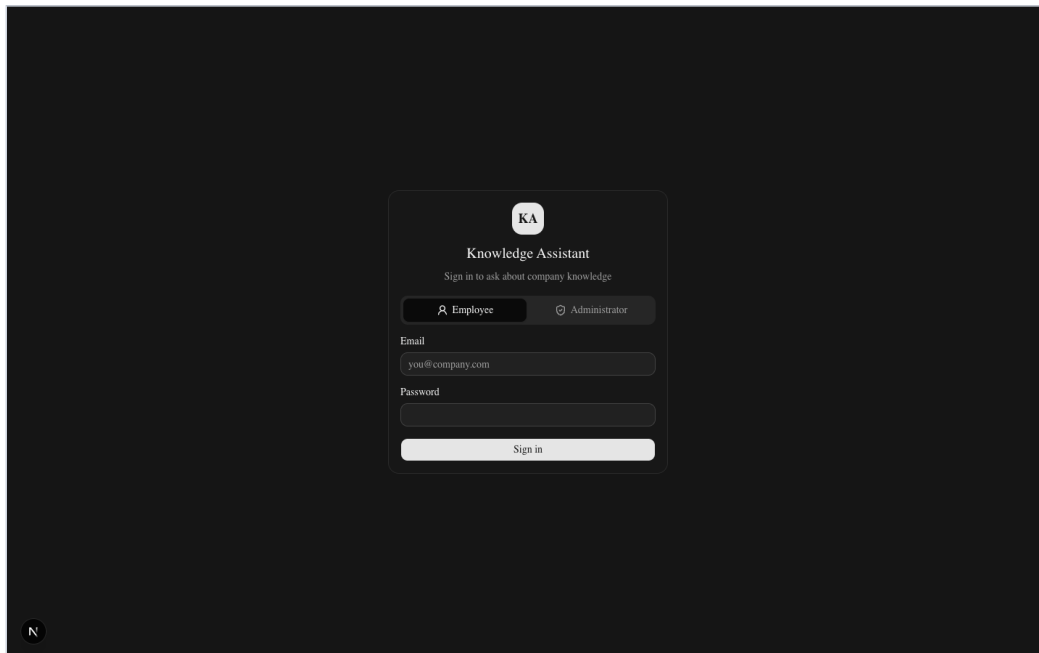
Improvement	Reason
Rebuild the Android APK and macOS image	Ship the cleartext fix and current screens on every platform.
Code signing and notarisation	Remove first-launch warnings.
Commit history and CI on pull requests	Record authorship and run tests automatically.
Frontend and end-to-end tests	Cover portals, chat streaming and admin flows in a browser.
Answer-quality evaluation	Measure retrieval and answer accuracy with real providers on a fixed question set.
httpOnly cookie sessions	Reduce exposure of refresh tokens.
Local PostgreSQL option	Match production behaviour during development.
Object storage	Implement the FileStorage protocol for S3-compatible storage.
Additional model provider	Add a second LLMClient implementation.
Production network profile	TLS reverse proxy and restricted CORS for real deployments.

20. Conclusion

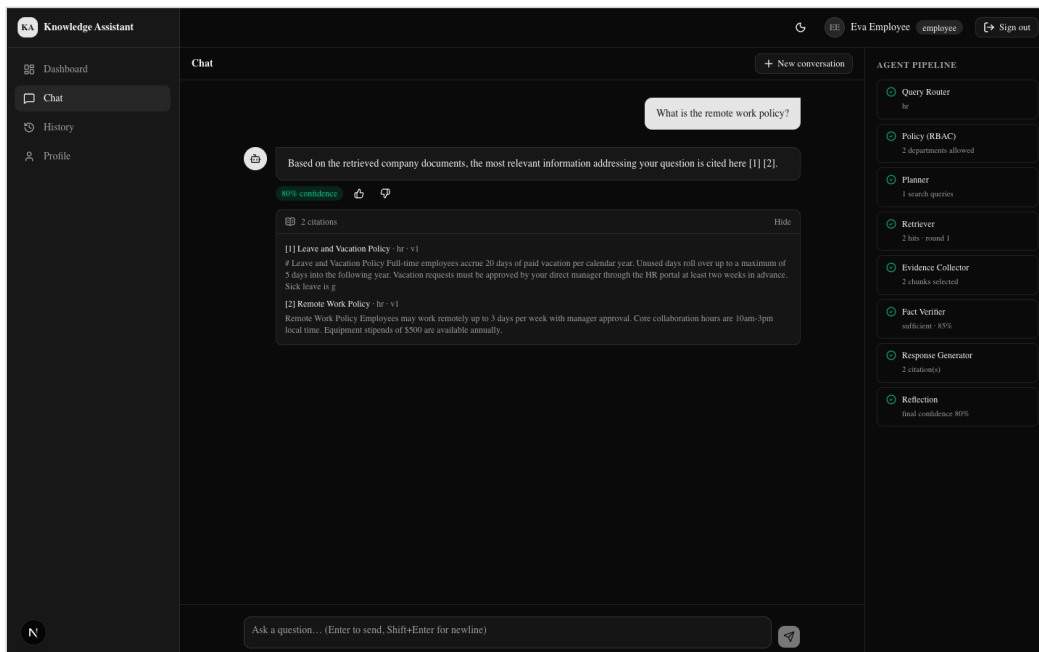
The project delivers a working, tested platform for answering questions from an organisation's documents with verifiable sources and department-level access control. Its central design choices – access policy and evidence handling in code, a bounded agent graph, providers behind interfaces and background ingestion – keep behaviour predictable and testable without external services. The later work moved it from a single macOS machine to a Windows workstation serving several devices, and fixed defects that only appeared in real use. The remaining work is mainly on distribution: rebuilding and signing the mobile and macOS clients, and adding browser-level tests.

Appendix A – Interface Walkthrough

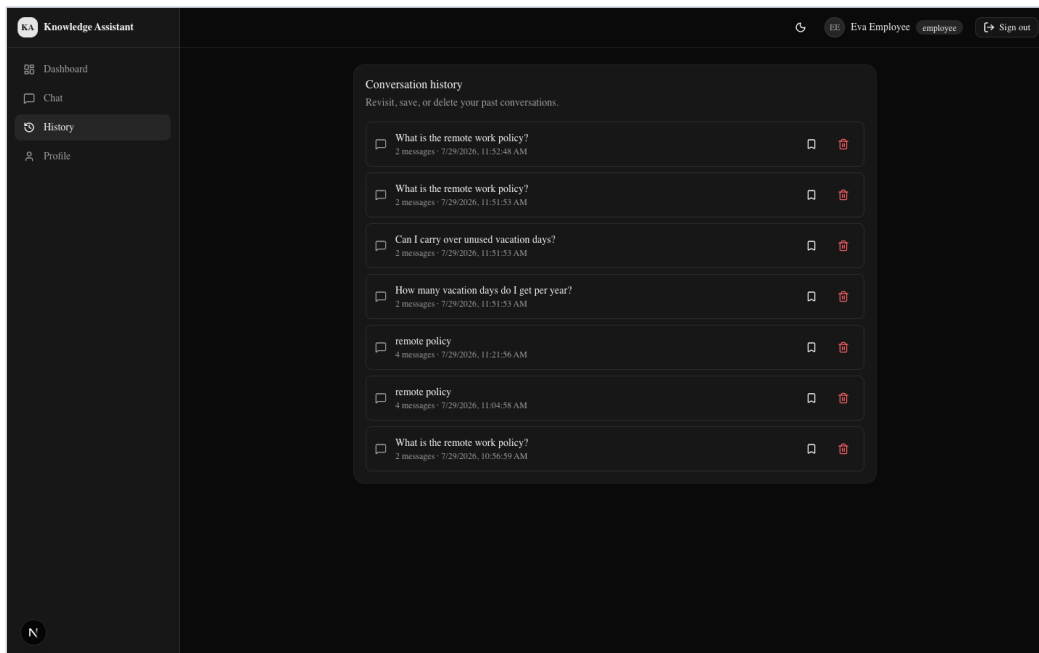
Screens captured from the running application at an earlier build. Some have since gained features (invitations and the edit sign-in details dialog on Users; email and password forms on Profile).



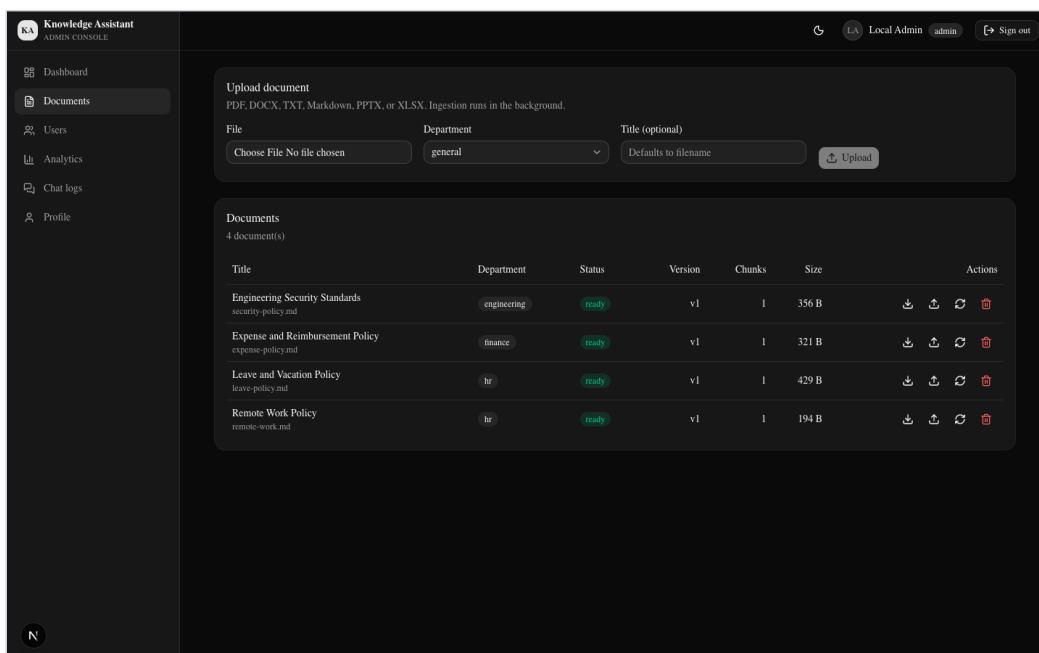
A1 – Sign-in with the Employee / Administrator portal selector.



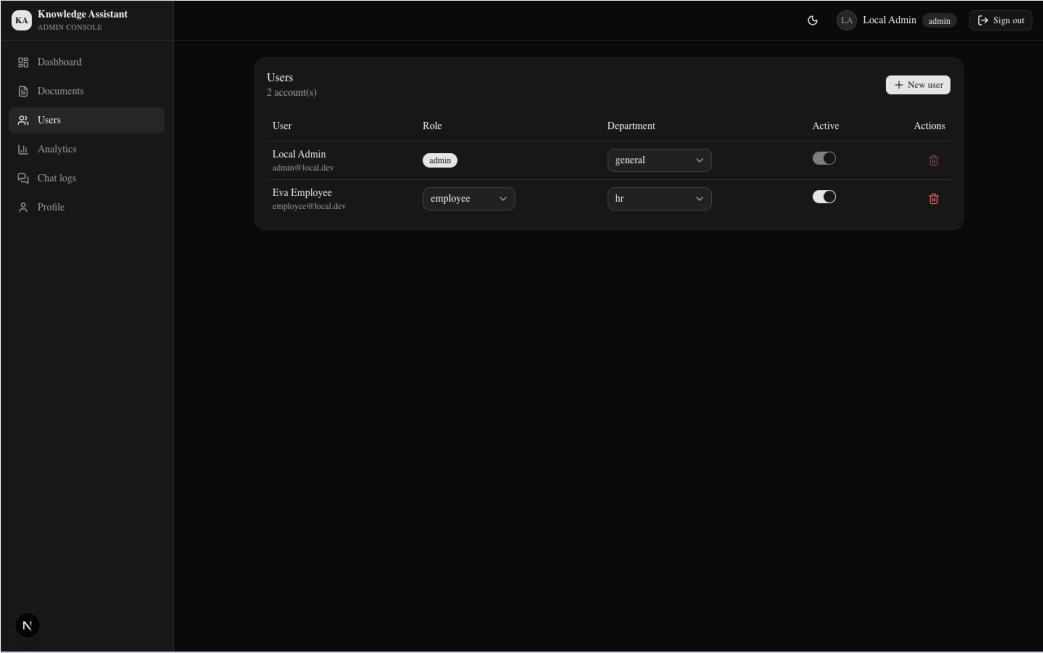
A2 – Chat: answer with confidence badge, citations and the agent pipeline panel.



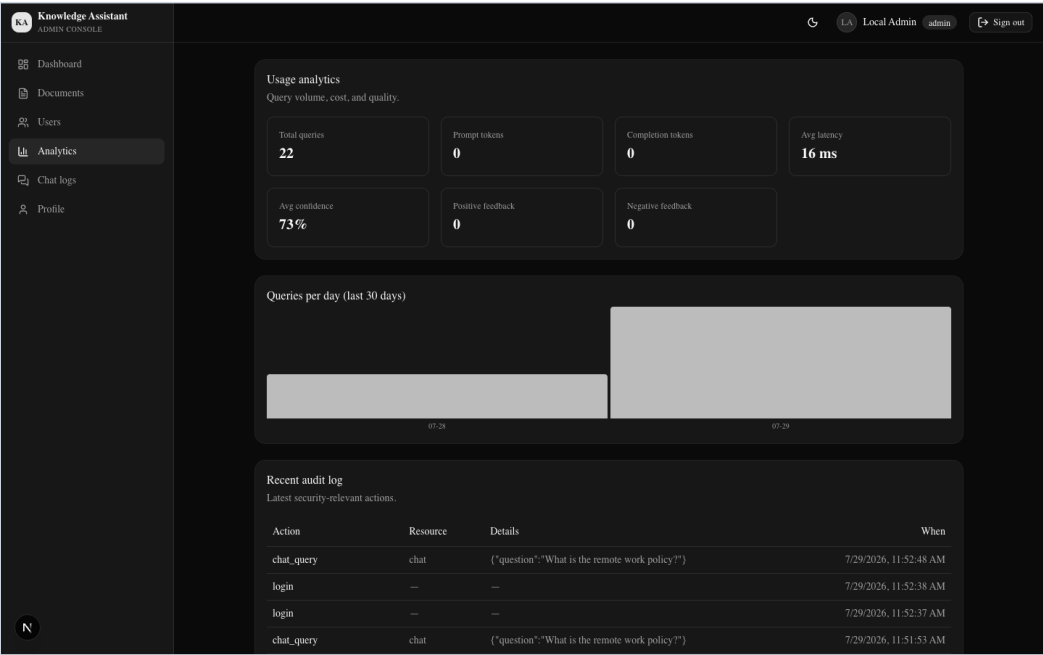
A3 – Conversation history.



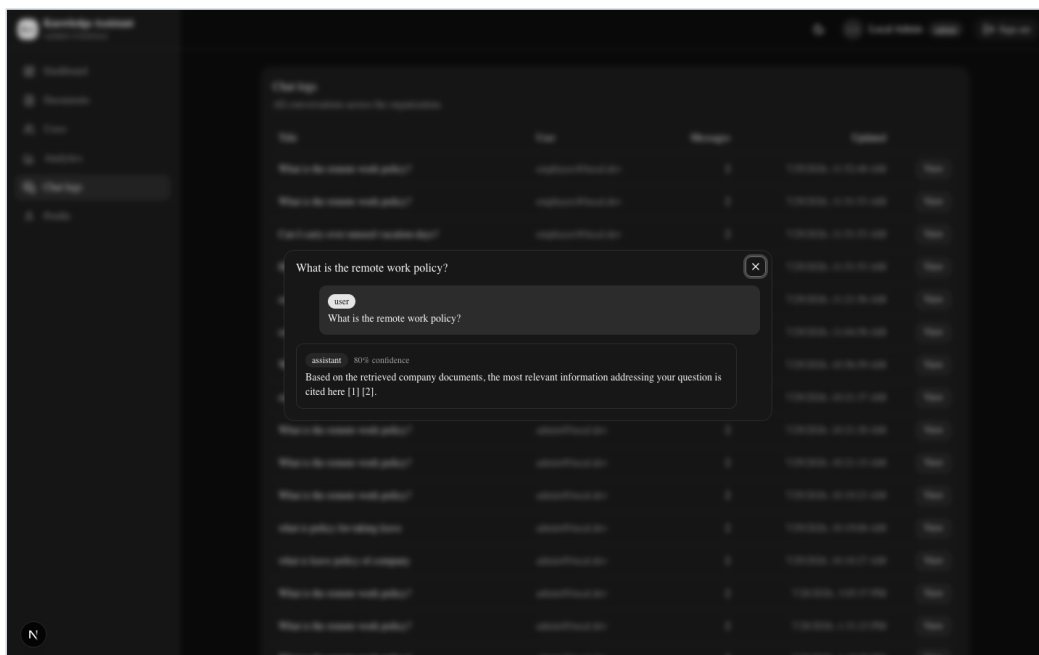
A4 – Administrator documents: upload form, status, versions and chunk counts.



A5 – Administrator users: role, department and active state.



A6 – Analytics: queries, tokens, latency, confidence and feedback.



A7 – Read-only transcript from the chat logs.

Appendix B – API Reference

All paths are under /api/v1. Access: public, user (signed in), owner, admin.

Method	Path	Access	Purpose
GET	/health, /health/ready	public	Liveness; readiness (database reachable)
POST	/auth/login, /auth/refresh	public	Token pair; rotate tokens
GET	/auth/me	user	Current profile
PATCH	/auth/me/email, /auth/me/password	user	Self-service changes (current password required)
GET	/auth/invite/{token}	public	Preview an invitation
POST	/auth/invite/accept	public	Create the account and sign in
POST	/ask	user	One-off question, not stored
POST	/chat, /chat/stream	user	Question in a conversation; streamed agent events
GET	/conversations	owner	List conversations with message counts
GET / PATCH / DELETE	/conversations/{id}	owner	Transcript; rename or save; delete
POST	/feedback	owner	Rate a message (upsert)
GET / POST	/documents	admin	List (filterable); upload (202)
GET / PUT / DELETE	/documents/{id}	admin	Detail; replace with new version; delete
GET	/documents/{id}/download	admin	format = original, pdf, docx, txt, md
POST	/documents/{id}/reindex	admin	Re-run ingestion
GET / POST	/users	admin	List; create
GET / PATCH / DELETE	/users/{id}	admin	Detail; update (incl. email, password); deactivate
GET / POST	/invitations	admin	List; issue a link
DELETE	/invitations/{id}	admin	Revoke a pending invitation
GET	/admin/chat-logs, /admin/chat-logs/{id}	admin	All conversations; any transcript
GET	/admin/analytics, /admin/audit-logs	admin	Usage rollups; audit trail

Appendix C – Master Prompt

C.1 How to use

C.2 is a single build specification for the whole project. C.3 wraps it in the prompt structure each model family expects. Choose one structure, paste the complete C.2 text where it says so, and give the prompt to a coding agent with file-system and shell access in an empty directory with Python 3.12+ and Node.js 20+ installed.

C.2 Build specification

BUILD SPECIFICATION - ENTERPRISE MULTI-AGENT KNOWLEDGE ASSISTANT

1. PRODUCT

A private, self-hosted question-answering platform over one organisation's documents. Administrators publish documents; employees ask questions in natural language and receive answers that cite the source document, department, version and page, with a confidence score. Answers come from an 8-agent LangGraph pipeline that plans, retrieves, verifies and self-checks before responding. Department-level access control is enforced in code, never by the model. One codebase ships as: a web app, a Windows/macOS desktop app (Electron) and an Android app (Capacitor). Packaged apps contain only the UI and talk to the organisation's backend over HTTP.

2. TECHNOLOGY (use exactly these)

Backend	Python 3.12 FastAPI Uvicorn[standard] Pydantic v2 + pydantic-settings SQLAlchemy 2.0 async asyncpg aiosqlite Alembic LangGraph openai SDK qdrant-client PyJWT bcrypt email-validator structlog python-multipart pytest + pytest-asyncio + httpx
Ingestion	pypdf python-docx python-pptx openpyxl langchain-text-splitters Export: fpdf2 python-docx
Frontend	Next.js 16 (App Router) React 19 TypeScript strict Tailwind CSS 4 shadcn/ui on Base UI next-themes lucide-react sonner
Data	PostgreSQL 16 (production), SQLite via aiosqlite (local dev/tests) Qdrant: server (production) or embedded "local:<path>" / ":memory:"
Clients	Electron 33 + electron-builder (NSIS .exe, .dmg, AppImage) Capacitor 8 (Android .apk; iOS via CI)
Deploy	Docker + Docker Compose GitHub Actions release matrix optional LangSmith tracing

3. ARCHITECTURE RULES

- Layers: api/ (HTTP only) -> services/ (business logic) -> repositories/ (data access). graph/ (agents) is called by services and never touches the database. Repositories never commit; the request-scoped session commits on success and rolls back on any exception.
- Application factory create_app(settings); no module-level mutable state; everything injected with FastAPI Depends and stored on app.state in the lifespan handler.
- All configuration from environment (pydantic-settings, .env, .env.example).
- One AppError hierarchy (NotFound 404, Conflict 409, Unauthorized 401, Forbidden 403, Validation 422) mapped to {"error":{"code","message"}}; RequestValidationError uses the same envelope (code "validation_error").
- structlog: console in development, JSON otherwise.
- Full type hints; a Pydantic schema for every request and response.

4. DATA MODEL - 9 tables, UUID primary keys, timezone-aware timestamps

users	email(unique, lower-cased), hashed_password, full_name, role(admin employee), department, is_active
documents	filename, title, department, version, status(pending processing ready failed), file_path, mime_type, size_bytes, chunk_count, error_message, uploaded_by_id(SET NULL)
conversations	user_id(CASCADE), title, is_saved
chat_messages	conversation_id(CASCADE), role(user assistant), content, citations JSON, confidence, token_usage JSON
feedback	message_id, user_id, rating(positive negative), comment
permissions	user_id, department, granted_by_id; UNIQUE(user_id,department)

audit_logs user_id, action, resource_type, resource_id, details JSON, ip_address, created_at (append-only)

analytics_events event_type, user_id, prompt_tokens, completion_tokens, latency_ms, properties JSON, created_at

invitations email, full_name, role, department, token_hash(sha256), invited_by_id, expires_at, accepted_at, created/updated_at

Enums: UserRole(admin, employee); Department(hr, finance, engineering, legal, sales, general); DocumentStatus; MessageRole; FeedbackRating; AuditAction (login, logout, document_upload/delete/reindex, user_create/update/delete, user_invite, user_invite_revoke, user_invite_accept, chat_query).

Conventions:

- JSON columns use a JSONB-on-PostgreSQL / JSON-elsewhere variant.
- Explicit constraint naming convention for deterministic migrations.
- TIMESTAMPS: every created_at/updated_at gets a client-side default (default=datetime.utcnow, onupdate for updated_at) AND a server default. In migrations write server defaults as sa.func.now(), NEVER sa.text('now()'): text() is emitted verbatim and SQLite has no now() function, so inserts that rely on it fail with "unknown function: now()".
- New enum values need a hand-written migration: ALTER TYPE <type> ADD VALUE IF NOT EXISTS '<NAME>' inside an autocommit block, skipped on SQLite. Models persist the enum NAME.

5. MULTI-AGENT PIPELINE (LangGraph)

START -> router -> policy -> planner -> retriever -> evidence -> verifier
 verifier: sufficient OR budget spent -> generator, else -> retriever
 generator -> reflection
 reflection: needs_retrieval AND budget left -> retriever, else -> END
 Both loops share ONE counter (MAX_AGENT_ITERATIONS, default 2) so the graph always terminates.

router	LLM	1-2 departments (structured output limited to the enum); "general" = not clearly one department.
policy	CODE	allowed = own department + general + granted departments (admins: all). permitted = routed intersect allowed. If empty -> allowed. If "general" is among permitted -> allowed (general is a catch-all, not a filter).
planner	LLM	2-4 plan steps and 1-3 search queries as noun phrases.
retriever	CODE	embed queries; department-filtered Qdrant search, top_k 8. First round uses permitted departments; ANY retry round widens to allowed departments (never repeat the same narrow search). allowed is always the ceiling. Increments iteration.
evidence	CODE	keep newest version per document, drop hits below the score floor, dedupe by (document_id, chunk_index) keeping the best score, top 6 by score.
verifier	LLM	sufficient, confidence 0-1, issues, optional refined query.
generator	LLM	answer ONLY from numbered evidence with inline [n] markers; returns cited indexes; citations are BUILT IN CODE from those indexes (name, department, version, page, source, 300-char snippet, score), ignoring out-of-range indexes.
reflection	LLM	is_complete, has_citations, needs_retrieval, final_confidence.

State (TypedDict): query, user_role, user_department, granted_departments, allowed_departments, routed_departments, plan, search_queries, retrieved, evidence, verification, reflection, iteration, answer, citations, confidence, token_usage (additive reducer so total cost is exact).

Providers behind protocols:

LLMClient.structured(schema, system, user) -> (model, token_usage)
 OpenAILLMClient (structured outputs, temperature 0, max_retries 3)
 FakeLLMClient (deterministic keyword heuristics; prompts carry "QUESTION:" and "EVIDENCE COUNT:" lines it can read)
 EmbeddingProvider with OpenAI and Fake (hash-based, normalised) versions.
 Selected by LLM_PROVIDER / EMBEDDING_PROVIDER = openai | fake.
 Adding another vendor must be one new class, no agent changes.

6. INGESTION

Upload -> validate -> store -> extract -> chunk -> embed -> Qdrant -> ready

- Formats: pdf, docx, txt, md, pptx, xlsx. Loaders return sections with page (pdf page / pptx slide / xlsx sheet) provenance.
- Reject unsupported extensions, empty files, files over MAX_UPLOAD_SIZE_MB (50). Store under a UUID-prefixed name; strip client path components; FileStorage protocol (LocalFileStorage) with a path-escape guard.
- RecursiveCharacterTextSplitter 1000/150, provenance on every chunk.

- Qdrant collection "documents", cosine, keyword payload indexes on department and document_id. Payload: document_id, document_name, department, version, chunk_index, page, source, uploaded_at, text.
- Runs as a FastAPI background task with its OWN session. The document row is the progress record: pending -> processing -> ready | failed (+ message).
- Re-index and version replacement delete the document's vectors before upserting.

7. AUTH, ACCESS CONTROL, ACCOUNTS

- bcrypt; JWT HS256 access (30 min) and refresh (7 days) with a type claim and a jti; each type rejected where the other is expected.
- Login: identical "Incorrect email or password." for unknown email and wrong password, verified against a dummy hash for timing parity. Inactive users rejected on every request.
- Guards: admin-only routes; owners only for conversations and feedback (others get 404); policy agent before retrieval.
- Two portals on one login screen (Employee / Administrator selector). The account role must match the chosen portal, else show an explicit message.
Employee: Dashboard, Chat, History, Profile.
Administrator: Dashboard, Documents, Users, Analytics, Chat logs, Profile.
Administrators do not chat. Route guards in both directions.
- No public sign-up. Admins add people by:
 - a) Invitation: random token (token_urlsafe(32)) shown ONCE, only its sha256 stored; default 7 days; single use; role and department fixed by the invite; re-inviting replaces the old link; pending links revocable; accepted ones not. Accepting creates the account and signs in.
 - b) Direct creation with a temporary password.
- Credentials:
 - Admin: PATCH /users/{id} may change email (lower-cased, 409 if taken) and password without the old one.
 - Self-service: PATCH /auth/me/email and PATCH /auth/me/password require the current password. A wrong current password returns 422, NOT 401 (the web client treats 401 as an expired session and would sign the user out).
- Admins cannot demote, deactivate or delete themselves.
- Audit: logins, uploads, deletes, re-indexes, user changes, invitations, queries. Credential changes record which fields changed and by whom ("admin"/"self"), never the values.

8. REST API - 36 operations under /api/v1

```

GET /health          GET /health/ready
POST /auth/login      POST /auth/refresh      GET /auth/me
PATCH /auth/me/email PATCH /auth/me/password
GET /auth/invite/{token} POST /auth/invite/accept
POST /ask             POST /chat             POST /chat/stream
GET /conversations    GET|PATCH|DELETE /conversations/{id}
POST /feedback        (upsert per user+message)
GET|POST /documents   GET|PUT|DELETE /documents/{id}
GET /documents/{id}/download?format=original|pdf|docx|txt|md
POST /documents/{id}/reindex
GET|POST /users       GET|PATCH|DELETE /users/{id} (DELETE = deactivate)
GET|POST /invitations DELETE /invitations/{id}
GET /admin/chat-logs  GET /admin/chat-logs/{id}
GET /admin/analytics  GET /admin/audit-logs
POST /documents returns 202 and ingests in the background.
/chat/stream (text/event-stream, "data: {json}"):
{"type":"conversation",...} then one {"type":"agent","agent":<name>,
"info":{...}} per completed agent, then {"type":"done","message":{...}}.
info summaries: router departments, policy allowed, planner queries,
retriever hits + round, evidence selected, verifier sufficient +
confidence, generator citations, reflection final confidence.

```

Export: "original" streams stored bytes; pdf/docx/txt/md are generated from the same extraction as ingestion (text only; PDF core fonts are Latin-1, so encode with errors="replace").

9. FRONTEND

- lib/types.ts mirrors every schema. lib/api.ts: typed client, bearer token, ONE shared in-flight refresh on 401 then retry once, else clear session and go to /login. Wrap fetch so an unreachable server raises
"Cannot reach the API server at <url>. Make sure the backend is running."
- lib/config.ts: API base = localStorage "eka.api_base" OR build-time NEXT_PUBLIC_API_BASE (default http://localhost:8000/api/v1). If that default is a loopback host, the page is NOT a packaged app, and the page was

- opened from another host (e.g. `http://192.168.x.x:3000`), substitute the page's hostname. `normalizeApiBase` adds `http://` and `/api/v1` when missing; `checkServer` probes `/health` before saving.
- `next.config.ts`: output "standalone", or "export" with trailingSlash and unoptimized images when `APP_SHELL=1`; `allowedDevOrigins` = this machine's non-internal IPv4 addresses (`os.networkInterfaces`) so the dev UI works from other devices.
- Pages: `/login` (portal selector + server panel: Change / Test & save), `/invite` (accept), `/dashboard`, `/chat`, `/history`, `/profile` (details + change email + change password, both needing the current password), `/admin/documents`, `/admin/users`, `/admin/analytics`, `/admin/chat-logs`.
- Chat: `?c=` threading, optimistic bubbles, confidence badge (green $\geq 70\%$, amber $\geq 40\%$, red below), expandable citations, thumbs feedback, live Agent Pipeline panel (8 agents: pending / running / done with result, xN retries) driven by `/chat/stream`.
- Admin documents: upload form, status table, poll while pending/processing, download menu (5 formats via fetch + blob so the token is sent), replace, re-index, delete. Admin users: invite dialog (link shown once, copy button; when `window.location` is not `http(s)` show the path to append instead), create dialog, inline role/department, active switch, pencil icon opening "Edit sign-in details" (email + new password + confirm).
- `shadcn/ui` here is built on Base UI: compose with `render={<X />}` (no `asChild`); `nativeButton={false}` for link buttons; menu labels inside `DropdownMenuGroup`; `Select` on `ValueChange` gives string | null.
- Wrap `useSearchParams` users in `Suspense`. Read `node_modules/next/dist/docs` before using any Next.js API.

10. PACKAGED CLIENTS

- `desktop/main.js`: serve the static export from an in-process HTTP server on 127.0.0.1 (random port, path-traversal guard, SPA fallback), open `/login/`, set `window.__EKA_DESKTOP__ = true`, open external links in the system browser, platform-aware menu.
- `desktop/sync.mjs` (Node, not shell): run the frontend build with `APP_SHELL=1`, copy out/ to `web/`. Refuse to run when port 3000 is listening (next build and next dev share `.next`). Scripts: `build:win` (NSIS x64, not one-click, desktop shortcut), `build:mac` (dmg arm64+x64), `build:linux`.
- Capacitor: `appId` `com.enterprise.knowledgeassistant`, `webDir` out, `androidScheme` `https`, `allowMixedContent` `true` AND `android:usesCleartextTraffic="true"` in `AndroidManifest` (`allowMixedContent` alone does not permit `http://` servers on `targetSdk 28+`).
- Bake a customer server with `NEXT_PUBLIC_API_BASE` at build time.
- CI (`release.yml`): desktop matrix on macOS/Windows/Ubuntu, Android APK, unsigned iOS, publish to GitHub Releases.

11. LOCAL RUN TOOLING (Windows)

- `setup.ps1` + `setup.cmd`: find Python 3.12+ (py launcher, PATH, per-user install; ignore the Microsoft Store stub), rebuild a non-Windows virtualenv, pip install, write `backend/.env` with SQLite + embedded Qdrant + fake providers + generated JWT secret (never overwrite an existing `.env`), alembic upgrade head, optional admin, reinstall macOS `node_modules`, write `frontend/.env.local`. Idempotent.
- `start.ps1` (+ `start.cmd`, which passes `-Lan` and bypasses execution policy): `-Lan` binds 0.0.0.0; checks the PC still has the address baked into the apps; if the firewall rule "Knowledge Assistant (dev)" (TCP 8000,3000, Private+Domain) is missing, requests elevation ONCE via RunAs and never prompts for input; refuses to start if a port is busy or another backend holds the single-writer embedded Qdrant store; `uvicorn --reload --reload-dir app`; next dev bound to 127.0.0.1 unless `-Lan`; waits for `/health`; prints the address for other devices; on exit kills process `TREES` (`taskkill /T /F`) and sweeps `node/python` processes whose command line contains this repository path or "uvicorn app.main" - never other projects' processes. `KA_SKIP_FIREWALL=1` skips the firewall step for tests.
- CORS: allow localhost, 127.0.0.1, capacitor://localhost and private networks (10/8, 172.16/12, 192.168/16, 100.64/10) on any port; never public origins. Overridable by `CORS_ORIGINS` / `CORS_ORIGIN_REGEX`.

12. DEPLOYMENT

`backend/Dockerfile` multi-stage, non-root, curl HEALTHCHECK, uvicorn 2 workers. Entrypoint waits for the database, runs alembic upgrade head, seeds an admin from `INITIAL_ADMIN_*` if set. `frontend/Dockerfile` 3-stage Node 22 alpine standalone, non-root, `NEXT_PUBLIC_API_BASE` as build arg.

`docker-compose`: postgres:16-alpine with `pg_isready` health gate, qdrant, backend,

frontend; named volumes; required secrets via \${VAR:?message}.
Refuse to start in production with the default JWT secret; disable /docs in production. scripts/create_admin.py (creates or PROMOTES to admin) and scripts/reset_password.py (changes only the hash, keeps the role).

13. TESTS - no external services

SQLite file DB per test, Qdrant ":memory:", fake providers. Suites: health, db, auth, users, user_credentials, invitations, documents, graph, ask, chat, enum_migrations (parametrised: every persisted enum member must appear in a migration). Must cover RBAC isolation (HR never sees finance), vague-query widening, retry widening, graceful degradation on an empty knowledge base, SSE emitting all 8 agent events, ownership 404s, invitation single-use and expiry, credential changes and their audit entries, all 5 downloads.
Expected: 108 passing. Frontend: tsc --noEmit, eslint, next build.

14. BUILD ORDER (finish and verify each before the next)

1 Foundation 2 Database + migrations 3 Auth + users 4 Ingestion
5 Agent graph + /ask 6 Chat, feedback, admin reporting 7 Frontend
8 Docker 9 SSE + Agent Pipeline UI 10 Portal separation 11 Export
12 Invitations 13 Credential management 14 Desktop + Android shells
15 Windows tooling + multi-device access

For each: what was built, files created/changed, commands, test results, known issues. Then stop for approval.

C.3 Prompt structures

Claude

```
<role>
You are a senior AI software architect and full-stack engineer who ships
production-quality, fully tested systems.
</role>

<context>
Build, from an empty directory, a self-hosted enterprise knowledge assistant:
administrators publish company documents, employees ask questions and get
cited, confidence-scored answers from an 8-agent LangGraph pipeline with
department access control enforced in code. It ships as a web app plus
Windows/macOS/Android clients that connect to the same backend.
</context>

<instructions>
1. Read the complete build specification in <input> before writing code.
2. Work milestone by milestone in the order the specification gives.
3. For each milestone, write the code and its tests, run them, and fix
   failures before moving on.
4. Read the installed Next.js docs (node_modules/next/dist/docs) before using
   any Next.js API.
5. Report each milestone in the format below, then stop for approval.
</instructions>

<constraints>
- Use exactly the listed technologies and versions.
- No placeholder code, broken imports or TODO stubs in finished milestones.
- Never hardcode secrets; configuration comes from the environment.
- Access control must never depend on model output.
- Every backend test must run with no network, database server or API key.
- Apply every rule under "Conventions", the policy/retriever rules and the
  pitfalls exactly as written.
</constraints>

<input>
[Paste the complete BUILD SPECIFICATION from Appendix C.2 here.]
</input>

<examples>
<example>
Input: Milestone 5 - Agent graph + /ask
Output: Files created (graph/state.py, graph/llm.py, graph/nodes/*.py,
graph/builder.py, api/v1/routes/ask.py, tests/test_graph.py,
tests/test_ask.py); commands (pytest tests/test_graph.py tests/test_ask.py -v);
result (13 passed); known issues (none); next milestone (6).
</example>
<example>
Input: A vaguely worded HR question routed to "general"
Output: The policy agent widens scope to all departments the user may read,
so the user's HR documents are still searched and cited.
</example>
</examples>

<output_format>
Per milestone:
1. What was built and why
2. Folder structure (changed parts)
3. Files created
4. Files modified
5. Commands to run
6. Test results (actual output summary)
7. Known issues
8. Next milestone
</output_format>

<final_check>
Before finishing, confirm: 36 API operations exist; 9 tables migrate on
PostgreSQL and SQLite; 108 backend tests pass; the frontend builds with strict
```

TypeScript; start.cmd starts both servers without asking for input; the web UI works when opened from another device on the network.

```
</final_check>
```

OpenAI

Role: Senior full-stack engineer building a self-hosted, multi-agent document question-answering platform end to end, with repository and shell access.

Personality

Precise, methodical and candid. Verifies before claiming. Reports failures with their output. No filler.

Goal

A working repository matching the BUILD SPECIFICATION (Appendix C.2): FastAPI backend with an 8-agent LangGraph pipeline, Next.js frontend, Electron and Capacitor clients, Windows run scripts and Docker deployment.

Success criteria

- All 36 API operations implemented under /api/v1.
- 9 tables created by Alembic on PostgreSQL and SQLite.
- 108 backend tests pass with no external services.
- tsc --noEmit, eslint and next build pass.
- Department RBAC is enforced in code; an HR employee never receives finance content.
- start.cmd starts both servers with no prompts; other devices on the network can use the web UI and the apps.

Constraints

- Use only the listed technologies.
- No secrets in code; configuration via environment variables.
- Follow every convention, agent rule and pitfall in the specification.
- Build milestones in order; do not start one until the previous passes.

Output

For each milestone: summary, files created/modified, commands, test results, known issues, next step. Plain Markdown, concise.

Stop rules

- A test fails: fix it and rerun before continuing.
- A library API differs from expectations: read its installed docs, adapt, and note it.
- A requirement is ambiguous in a way that changes behaviour: ask.
- After each milestone report: stop and wait for approval.

BUILD SPECIFICATION

[Paste Appendix C.2 here.]

GPT-4.1

Role and Objective

You are a senior full-stack AI engineer. Build the Enterprise Multi-Agent Knowledge Assistant exactly as specified in the Context section.

Instructions

- Implement the system milestone by milestone.
- Write tests with every milestone and run them before continuing.
- Follow every rule in the specification literally; do not simplify the architecture.

Sub-categories

Backend

FastAPI layered api/services/repositories, async SQLAlchemy, Alembic migrations using sa.func.now() defaults, JWT auth, invitations, credential management, audit logging.

Agent pipeline

8 LangGraph agents, one shared iteration budget, policy and retriever widening rules, citations built in code, OpenAI and fake providers.

Frontend and clients

Next.js 16 portals, typed API client with shared refresh, runtime server

```

configuration, Electron and Capacitor shells, cleartext allowed on Android.
### Operations
Windows setup/start scripts with network mode and one-time firewall rule,
Docker Compose, CI release matrix.

# Reasoning Steps
1. Read the whole specification and list the milestones.
2. For the current milestone, list files to create and tests to write.
3. Implement, run tests, fix failures.
4. Verify the milestone against the specification.
5. Report and stop.

# Output Format
Milestone N - <name>
- Built:
- Files created:
- Files modified:
- Commands:
- Test results:
- Known issues:
- Next:

# Examples
## Example 1
Input: "Implement the policy agent."
Output: A pure function computing allowed departments (admins get all), a
node that intersects the router's choice with it, falls back to all allowed
when empty, and widens to all allowed when "general" is chosen, plus tests
proving an HR employee never reaches finance documents.

# Context
[Paste the BUILD SPECIFICATION from Appendix C.2 here.]

# Final instructions
Think step by step. Do not end a milestone until its tests pass. The finished
project must have 36 API operations, 9 tables and 108 passing backend
tests.

```

Codex

Objective

Build the Enterprise Multi-Agent Knowledge Assistant from an empty repository.

Context

A self-hosted platform where administrators publish documents and employees get cited, confidence-scored answers from an 8-agent LangGraph pipeline. Delivered as a web app plus Windows, macOS and Android clients.

Current State

Empty directory. Python 3.12+, Node.js 20+ available. PostgreSQL optional (SQLite is used locally and in tests).

Requirements

- * Implement everything in the BUILD SPECIFICATION (Appendix C.2).
- * backend/: FastAPI app, 9 models, 3+ Alembic migrations, services, repositories, graph/, ingestion/, scripts/, tests/.
- * frontend/: Next.js 16 app with the 11 listed pages and the typed client.
- * desktop/: Electron main.js, sync.mjs, electron-builder config.
- * Root: setup.ps1/cmd, start.ps1/cmd, docker-compose.yml, README.md, .github/workflows/release.yml.

Constraints

- * Exact technologies from the specification.
- * api/ must not contain business logic; graph/ must not touch the database.
- * Migrations use sa.func.now() for timestamp server defaults.
- * No secrets committed; .env files git-ignored.
- * Android manifest sets usesCleartextTraffic="true".

Acceptance Criteria

- * 36 operations in the generated OpenAPI schema.
- * 108 backend tests pass.
- * HR employee cannot retrieve finance content; vague questions still find the

```

    user's own documents; the graph always terminates.
* Uploads reach status "ready" and produce citations.
* start.cmd starts both servers without any input prompt.

Verification
* Run: cd backend && python -m pytest -q
* Run: cd frontend && npx tsc --noEmit && npx eslint && npm run build
* Verify: POST /api/v1/documents then GET until status is ready.
* Verify: POST /api/v1/ask returns citations and confidence.
* Confirm: http://<lan-ip>:3000 loads and signs in from another device.

Deliverable
A complete repository matching the specification, with passing tests and a
README covering setup, running, portals, configuration and troubleshooting.

BUILD SPECIFICATION
[Paste Appendix C.2 here.]

```

Gemini

```

<role>
You are a helpful assistant acting as a senior full-stack AI engineer.
</role>

<instructions>
1. Build the project described in <context> milestone by milestone.
2. Write and run tests for each milestone before starting the next.
3. Follow every rule in the specification exactly.
4. After each milestone, report and wait for approval.
</instructions>

<constraints>
- Use only the technologies listed in the specification.
- Enforce access control in code, never through prompts.
- All backend tests must run without network, database server or API key.
- No hardcoded secrets.
</constraints>

<output_format>
For each milestone: what was built; files created and modified; commands;
test results; known issues; next milestone.
</output_format>

<context>
[Paste the BUILD SPECIFICATION from Appendix C.2 here.]
</context>

<task>
Create the complete Enterprise Multi-Agent Knowledge Assistant repository:
backend, frontend, desktop and Android shells, Windows run scripts, Docker
deployment and CI.
</task>

<final_instruction>
Do not report a milestone as complete until its tests pass. The finished
project must expose 36 API operations, migrate 9 tables and pass 108 backend
tests.
</final_instruction>

```

Universal structure

```

ROLE
Senior full-stack AI engineer with repository and shell access.

OBJECTIVE / GOAL
Recreate the Enterprise Multi-Agent Knowledge Assistant: cited,
confidence-scored answers over company documents from an 8-agent pipeline,
with department RBAC in code, on web, Windows, macOS and Android.

CONTEXT
Start from an empty directory with Python 3.12+ and Node.js 20+.

```

REQUIREMENTS / INSTRUCTIONS

Implement the BUILD SPECIFICATION (Appendix C.2) in its milestone order, writing and running tests for each milestone.

CONSTRAINTS

Exact technologies; layered architecture; no secrets in code; migrations with `sa.func.now()`; Android cleartext enabled; start scripts never wait for input.

SUCCESS CRITERIA

36 API operations; 9 tables; 108 backend tests passing; frontend type-check, lint and build clean; multi-device access works.

EXAMPLES

A vague HR question routed to "general" still searches every department the user may read; an HR employee never receives finance content.

OUTPUT FORMAT

Per milestone: built, files created/modified, commands, test results, known issues, next.

VERIFICATION / STOP CONDITIONS

Stop and fix on any failing test. Stop after each milestone for approval. Ask when an ambiguity would change behaviour.

BUILD SPECIFICATION

[Paste Appendix C.2 here.]